

noncopyable and nonmovable utility classes

Document #: P2895
Date: 2023-05-17
Project: Programming Language C++
Audience: LEWG
Reply-to: Sebastian Theophil
<stheophil@think-cell.com>
Jonathan Müller
<jmueller@think-cell.com>

Contents

- [1 Abstract](#)
- [2 Motivation](#)
- [3 Naming](#)
- [4 Proposed Implementation](#)
- [5 Acknowledgements](#)
- [6 Wording](#)
- [7 References](#)

§ 1 Abstract

We propose the addition of `std::noncopyable` and `std::nonmovable`, utility classes you can inherit from to make your class non-copyable and/or non-movable.

§ 2 Motivation

Developers may want to declare that a class is a move-only type, i.e., that it is movable but non-copyable. This is often useful for RAII classes that must not duplicate the resource they manage. Similarly, developers may want to declare that a class is neither movable nor copyable. Consider the following example that deals with a legacy API:

Before	After
<pre>```cpp extern "C" RegisterCallbackInLegacyAPI(void (* callback)(void const*), void const* context); static void callback(void const* context) noexcept; class legacy_callback { legacy_callback(legacy_callback const&) = delete; legacy_callback& operator=(legacy_callback const&) = delete; }</pre>	<pre>extern "C" RegisterCallbackInLegacyAPI(void (* callback)(void const*), void const* context); static void callback(void const* context) noexcept; class legacy_callback : std::nonmovable { void register_callback() noexcept { RegisterCallbackInLegacyAPI(callback, this); } }</pre>

Before	After
<pre> void register_callback() noexcept { RegisterCallbackInLegacyAPI(callback, this); } }; </pre>	<pre> }; </pre>

Passing the `this` pointer as context to a legacy callback requires that `this` remains constant. `std::mutex` is also an object that is neither copyable nor movable.

By declaring the copy-constructor and the copy-assignment operator of a class as deleted, developers can make their own classes non-copyable and non-movable. This seems straight-forward. Yet, users get this wrong. Indeed, two very popular StackOverflow answers on how to make an object non-copyable and non-movable were initially wrong, as the comments show. [1](#) [2](#).

Many libraries such as [boost](#) have introduced a type called `noncopyable` (or similar) that user-defined classes can derive from and that deletes both copy-construction and copy-assignment. Deriving from `noncopyable` lets users declare their *intent* clearly and succinctly. [3](#) [4](#) [5](#) [6](#) [7](#) [8](#)

A code search for “`noncopyable`” yields almost 5000 hits. It has thus clearly become a common idiom that is frequently used and reimplemented and therefore deserves to be included in the standard library.

§ 3 Naming

Unfortunately, `boost::noncopyable` (and all other referenced implementations except [8](#)) makes a class non-copyable **and** non-movable because the implementation of `boost::noncopyable` precedes the introduction of move semantics. This collides with the names of already standardized concepts `std::copyable` and `std::movable`. A class may not satisfy `std::copyable` yet may be `std::movable`. We propose to introduce types `noncopyable` and `nonmovable` that match the names of the already standardized concepts. Thus, a class deriving from `noncopyable` cannot satisfy `std::copyable` but may satisfy `std::movable`. A class deriving from `nonmovable` cannot satisfy `std::movable` and thus cannot satisfy `std::copyable` either.

Type `noncopyable` could also be called `moveonly` or `move_only`.

According to [codesearch.isocpp.org](#), the name `nonmovable` has only been used in the [test framework for the range v3 library](#) together with a type `moveonly`. The alternative spelling `move_only` has also often been used in test frameworks for `libcxx`, `libstdc++`, `boost.hana` and `gcc`.

§ 4 Proposed Implementation

We propose the following definitions for both classes:

```

struct noncopyable {
    noncopyable() = default;
    noncopyable(noncopyable&&) = default;
    noncopyable& operator=(noncopyable&&) = default;
};

struct nonmovable {
    nonmovable() = default;
    nonmovable(nonmovable const&) = delete;
    nonmovable& operator=(nonmovable const&) = delete;
};

```

In both classes, the default constructor is declared as default and is thus trivial. `noncopyable` and `nonmovable` are therefore empty standard layout classes and “empty base optimization” is required when deriving from either. Deriving from `noncopyable` or `nonmovable` therefore incurs no space or runtime overhead.

§ 5 Acknowledgements

We thank Alisdair Meredith for the original paper proposing to standardize the behavior of `boost::noncopyable`. [N2675].

§ 6 Wording

Add to header `<utility>` synopsis in 22.2.1 [utility.syn]

```
// [utility.noncopyable], class noncopyable
namespace noncopyable-adl-namespace {
    struct noncopyable;
}
using noncopyable-nonmovable-adl-namespace::noncopyable;

// [utility.nonmovable], class nonmovable
namespace nonmovable-adl-namespace {
    struct nonmovable;
}
using nonmovable-adl-namespace::nonmovable;
```

Append a new section to 22.2 [utility]

§ 22.2.x Support Classes [utility.support]

The following classes are provided to simplify the implementation of common idioms.

§ 22.2.x.1 Class `noncopyable` [utility.noncopyable]

```
namespace noncopyable-adl-namespace {
    struct noncopyable {
        noncopyable() = default;
        noncopyable(noncopyable&&) = default;
        noncopyable& operator=(noncopyable&&) = default;
    };
}
using noncopyable-adl-namespace::noncopyable;
```

- 1 `noncopyable` is provided to simplify creation of classes that have move-only semantics, i.e. they are movable but not copyable.

[Note: `noncopyable` is provided in an unspecified nested namespace to limit argument dependent lookup 6.5.4 [basic.lookup.argdep]; no other names should be declared in this namespace. — end note]

[Example:

```
class file : std::noncopyable {
public:
    file(std::string const& strPath)
        : fp(std::fopen(strPath.c_str(), "w"))
    {}

    file(file&& f) : fp(std::exchange(f.fp, nullptr)) {}
    file& operator=(file&& f) { ... }
```

```
    ~file() { if(fp) std::fclose(fp); }  
private:  
    std::FILE* fp;  
};
```

— *end example*]

§ 22.2.x.2 Class `nonmovable` [utility.nonmovable]

```
namespace nonmovable-adl-namespace {  
    struct nonmovable {  
        nonmovable() = default;  
        nonmovable(nonmovable const&) = delete;  
        nonmovable& operator=(nonmovable const&) = delete;  
    };  
}  
using nonmovable-adl-namespace::nonmovable;
```

- ¹ `nonmovable` is provided to simplify creation of classes that inhibit move and copy semantics.

[*Note*: `nonmovable` is provided in an unspecified nested namespace to limit argument dependent lookup 6.5.4 [basic.lookup.argdep]; no other names should be declared in this namespace. — *end note*]

[*Example*:

```
class mutex : std::nonmovable {  
public:  
    mutex();  
    ~mutex();  
};
```

— *end example*]

§ 7 References

[N2675] Alisdair Meredith. 2008-06-13. noncopyable utility class (revision 1).
<https://wg21.link/n2675>